Coinbase Maturity Relaxation under ChainLocks

Hilawe Semunegus, on an idea from Joel Valenzuela (TheDesertLynx)

The writeup for project D4 in dash-ideas-scoping.md, a small consensus change worth drafting now and attaching to the next hard fork rather than running on its own.

Plain words. When a miner finds a block, the coins it pays out cannot be spent for 100 blocks, about four hours. The rule is inherited from Bitcoin and exists for one reason. If the chain reorganizes and the block is orphaned, its reward never existed, and anyone paid with it downstream would be holding coins that vanished. Dash largely closed that door with ChainLocks, which let a masternode quorum sign each block within seconds and make deep reorganizations practically impossible. Once a block is ChainLocked, the danger the 100-block wait guards against is gone, so most of the wait is a leftover from a problem Dash already solved.

The rule today

Coinbase outputs are unspendable until they are 100 blocks deep. The constant is `COINBASE_MATURITY = 100` (`src/consensus/consensus.h:24`), enforced when a transaction tries to spend a coinbase output at `src/consensus/tx_verify.cpp:182` (reject reason `bad-txns-premature-spend-of-coinbase`) and again on mempool acceptance at `src/validation.cpp:493`. The purpose is reorg safety, a coinbase that gets orphaned must not have been spent onward.

Why ChainLocks change the calculus

ChainLocks (DIP-8) have a long-living masternode quorum sign the first-seen block at each height, within seconds of the block. A ChainLocked block cannot be reorganized out by a competing chain, because nodes reject any reorg that would remove a ChainLocked block. So the specific failure the maturity rule prevents, an orphaned coinbase whose reward was spent and then vanished, cannot happen to a ChainLocked block. A rule of the form "a coinbase output is spendable once its block is ChainLocked" is therefore coherent, and for a ChainLocked block it is strictly safer than waiting 100 blocks, since ChainLock finality arrives in seconds while 100 blocks is hours.

The one real design problem

ChainLock signatures are not part of block data. They are separate `clsig` messages that a running node verifies and holds, not something committed into the block or the coinbase. That is fine for a node online and following the tip, it has the ChainLocks. It is a problem for a node syncing from scratch, which downloads historical blocks but not necessarily the historical ChainLock signatures, and which therefore cannot locally prove that a two-year-old block was ChainLocked at the time. A consensus rule that makes spendability depend on ChainLock state has to answer how such a node validates a historical spend of a young coinbase. This is the piece that needs a concrete design before the change can ship, and it should be confirmed against the ChainLock implementation rather than assumed.

Options to explore, in rough order of increasing intrusiveness:

- Keep the 100-block rule as the consensus floor and treat ChainLock-based early spendability as wallet and relay policy only, so no historical-validation problem arises. This gives faster practical spendability with no consensus change, but it does not let a coinbase spend confirm into a block any earlier.
- Commit a compact ChainLock reference into a later block or the coinbase, so the finality proof travels with the chain and a syncing node can check it. This is a consensus change with its own design surface.
- Reduce the maturity constant instead of coupling it to ChainLocks, a blunter change that needs no historical proof but gives up the "seconds not hours" property.

Who benefits

The Dash coinbase pays more than the miner. Every block's coinbase pays the scheduled masternode alongside the miner under consensus enforcement, and treasury proposals are paid from superblock coinbases on their own cadence. So faster access to matured coinbase outputs helps miners, masternode operators, and funded treasury recipients, not just miners.

Recommendation

Small code, large process. The maturity rule is consensus, so any change is a hard fork, and a hard fork is an expensive coordination event regardless of how small the diff is. The sensible path is to settle the historical-validation question on paper, pick one of the options above, and attach the change to whichever hard fork ships next for other reasons rather than running a hard fork for this alone.

Where this sits

This is project D4 in the scoping document, formerly P2. It is the lowest-urgency of the four, useful and coherent, but not worth its own hard fork. The idea of applying ChainLock finality to coinbase spendability is Joel Valenzuela's.